

MAIE 5532 — Sample Midterm Exam 2 (60 minutes)

Weeks 1–5: Frameworks, AD, Graph Optimization, Parallelism, Distributed/Federated Learning

Instructions: Tick the box to the left of each chosen option. Some questions are multi-select.

Name: Type your name here

Student ID: Type your student ID here

Questions (40)

Question 1

Which AD mode most efficiently computes vector–Jacobian products for a scalar loss over millions of parameters?

- ☐ A. Forward-mode AD
- ☐ B. Reverse-mode AD
- ☐ C. Numerical finite differences
- ☐ D. Pure symbolic differentiation only

Knowledge point: Reverse-mode AD efficiently computes gradients for scalar losses with many parameters.

Slides: [Week 2 slides, p. 32](#); [Week 2 slides, p. 47](#) **Review PDF:** [All relevant slides, including other answer choices](#)

[Show answer & explanation](#)

Question 2

For dual numbers, a unary function obeys $f(a + \epsilon \cdot b) = ?$ (first-order exact)

- ☐ A. $f(a) + f'(a) \cdot \epsilon \cdot b$
- ☐ B. $f(a) + f'(a) \cdot b^2$
- ☐ C. $f(a) + f'(a) \cdot \epsilon \cdot b$
- ☐ D. $f(a)$ (no change)

Knowledge point: Dual-number evaluation follows the first-order Taylor rule.

Slides: [Week 2 slides, pp. 13–15](#) **Review PDF:** [All relevant slides, including other answer choices](#)

[Show answer & explanation](#)

Question 3

In TensorFlow Lite for Microcontrollers (TFLM), the core memory strategy for inference is:

- ☐ A. Rely on malloc/free for each inference

- ☐ B. Use the OS paging system to overcommit RAM
- ☐ C. Store all activations in external flash at runtime
- ☐ D. Provide a fixed tensor arena and avoid dynamic allocation in the hot path

Knowledge point: TFLM uses a fixed tensor arena instead of hot-path dynamic allocation.

Slides: [Week 1 slides, p. 55](#) **Review PDF:** [All relevant slides, including other answer choices](#)

[Show answer & explanation](#)

Question 4

In XLA-backed JIT compilation (e.g., JAX/TF), which is a primary benefit?

- ☐ A. Guaranteed zero memory usage
- ☐ B. Operation fusion, layout selection, and buffer reuse
- ☐ C. Eliminates the need for autograd
- ☐ D. Arbitrary dynamic shapes without retracing

Knowledge point: XLA JIT: operation fusion, layout selection, and buffer reuse.

Slides: [Week 3 slides, p. 37](#) **Review PDF:** [All relevant slides, including other answer choices](#)

[Show answer & explanation](#)

Question 5

Which linker option (used with -ffunction-sections/-fdata-sections) removes unused code/data from the final binary?

- ☐ A. -Os
- ☐ B. -flto
- ☐ C. -fno-plt
- ☐ D. --gc-sections

Knowledge point: Embedded code-size reduction with section garbage collection and linker options.

Slides: [Week 4 slides, p. 58](#) **Review PDF:** [All relevant slides, including other answer choices](#)

[Show answer & explanation](#)

Question 6

Which hierarchical all-reduce sequence is correct?

- ☐ A. Inter-node all-gather → intra-node reduce-scatter → inter-node reduce-scatter → intra-node all-gather
- ☐ B. Intra-node all-gather → inter-node all-gather → intra-node reduce-scatter → inter-node reduce-scatter
- ☐ C. Intra-node reduce-scatter → inter-node reduce-scatter → inter-node all-gather → intra-node all-gather

- ☐ D. Inter-node reduce-scatter → intra-node reduce-scatter → inter-node all-gather → intra-node all-gather

Knowledge point: The four communication stages of hierarchical all-reduce.

Slides: [Week 5 slides, p. 17](#) **Review PDF:** [All relevant slides, including other answer choices](#)

[Show answer & explanation](#)

Question 7

In secure aggregation for FL, handling client dropouts typically requires:

- ☐ A. Decrypting all updates at the server
- ☐ B. Replaying client updates from logs
- ☐ C. Homomorphic inference on raw audio
- ☐ D. Threshold mask-recovery so pairwise masks still cancel in the sum

Knowledge point: Secure aggregation dropout handling through threshold mask recovery.

Slides: [Week 5 slides, p. 50](#) **Review PDF:** [All relevant slides, including other answer choices](#)

[Show answer & explanation](#)

Question 8

In TensorFlow 2.x, the default execution style is:

- ☐ A. Eager (imperative) execution with optional tf.function
- ☐ B. Only static graphs with sessions
- ☐ C. Ahead-of-time compilation only
- ☐ D. TorchScript-style tracing only

Knowledge point: TensorFlow 2.x eager execution with optional tf.function.

Slides: [Week 1 slides, p. 35](#); [Week 1 slides, p. 49](#) **Review PDF:** [All relevant slides, including other answer choices](#)

[Show answer & explanation](#)

Question 9

TensorFlow 1.x's "define-then-run" programming corresponds to:

- ☐ A. Immediate execution
- ☐ B. Pure functional transformations
- ☐ C. Symbolic computational graphs executed in a Session
- ☐ D. Device-only kernels with no graphs

Knowledge point: TensorFlow 1.x define-then-run symbolic graphs and Session execution.

Slides: [Week 1 slides, p. 41](#) **Review PDF:** [All relevant slides, including other answer choices](#)

[Show answer & explanation](#)

Question 10

In TFLM, the FlatBuffer model format primarily enables:

- ☐ A. Online backprop on MCUs
- ☐ B. Zero-copy, schema-checked access without heavy parsing
- ☐ C. On-device code generation per model
- ☐ D. Dynamic operator linking at runtime

Knowledge point: FlatBuffers enable zero-copy, schema-based model access on MCUs.

Slides: [Week 1 slides, pp. 62–64](#) **Review PDF:** [All relevant slides, including other answer choices](#)

[Show answer & explanation](#)

Question 11

In TinyML systems, the dominant energy cost is often:

- ☐ A. ALU adds
- ☐ B. Branch prediction
- ☐ C. Stack frame setup
- ☐ D. Radio uplink (e.g., BLE/Wi-Fi)

Knowledge point: TinyML energy use, especially radio uplink cost.

Slides: [Week 4 slides, p. 35](#); [Week 6 slides, p. 26](#) **Review PDF:** [All relevant slides, including other answer choices](#)

[Show answer & explanation](#)

Question 12

A canonical fusion in CNN blocks that cuts memory traffic is:

- ☐ A. BatchNorm → Softmax → Dropout
- ☐ B. Pool → Pool → Pool
- ☐ C. Conv2D → BatchNorm → ReLU
- ☐ D. ReLU → Argmax → Sort

Knowledge point: Conv2D–BatchNorm–ReLU fusion reduces memory traffic and launches.

Slides: [Week 3 slides, pp. 21–22](#) **Review PDF:** [All relevant slides, including other answer choices](#)

[Show answer & explanation](#)

Question 13

For NHWC tensors, a cache-friendly inner loop typically iterates with:

- ☐ A. $h \rightarrow w \rightarrow c$ (c innermost)
- ☐ B. $c \rightarrow h \rightarrow w$ (c outermost)
- ☐ C. Randomized strides
- ☐ D. $h \rightarrow c \rightarrow w$ (w outermost)

Knowledge point: NHWC layout and channel-contiguous traversal.

Slides: Week 3 slides, pp. 25–26; Week 6 slides, p. 67 **Review PDF:** All relevant slides, including other answer choices

[Show answer & explanation](#)

Question 14

Which statement is not characteristic of synchronous all-reduce training?

- ☐ A. Per-step global barrier
- ☐ B. Weight staleness by design across workers
- ☐ C. Equivalent to a single device with the global batch
- ☐ D. Straggler sensitivity

Knowledge point: Synchronous all-reduce has barriers and straggler sensitivity, without designed-in staleness.

Slides: Week 5 slides, p. 16 **Review PDF:** All relevant slides, including other answer choices

[Show answer & explanation](#)

Question 15

A practical advantage of parameter-server (async/stale-sync) approaches is:

- ☐ A. No staleness
- ☐ B. Higher throughput and tolerance to slow clients
- ☐ C. Exact equivalence to single-device SGD
- ☐ D. Perfect reproducibility across runs

Knowledge point: Asynchronous parameter servers trade staleness for throughput and straggler tolerance.

Slides: Week 5 slides, pp. 15–16 **Review PDF:** All relevant slides, including other answer choices

[Show answer & explanation](#)

Question 16

In ZeRO Stage-2, the optimizer shards:

- ☐ A. Only parameter tensors
- ☐ B. Optimizer states and gradients
- ☐ C. Parameters and nothing else
- ☐ D. Nothing (just fuses kernels)

Knowledge point: ZeRO Stage 2 shards optimizer states and gradients.

Slides: [Week 5 slides, p. 18](#) **Review PDF:** [All relevant slides, including other answer choices](#)

[Show answer & explanation](#)

Question 17

In sharded training, the phase that computes partial reductions and partitions across ranks is:

- ☐ A. All-gather
- ☐ B. Broadcast
- ☐ C. All-to-all
- ☐ D. Reduce-scatter

Knowledge point: Reduce-scatter combines reduction with partitioning across ranks.

Slides: [Week 5 slides, pp. 17–18](#) **Review PDF:** [All relevant slides, including other answer choices](#)

[Show answer & explanation](#)

Question 18

If the TFLM tensor arena is too small at initialization, the correct behavior is to:

- ☐ A. Silently reallocate on the heap
- ☐ B. Compress the model in place
- ☐ C. Return an immediate error at init
- ☐ D. Stream tensors from flash automatically

Knowledge point: TFLM initialization must fail if the fixed arena is insufficient.

Slides: [Week 1 slides, p. 55](#) **Review PDF:** [All relevant slides, including other answer choices](#)

[Show answer & explanation](#)

Question 19

MLIR is best described as:

- ☐ A. A multi-level compiler framework with dialects enabling transformations from TF ops down to LLVM
- ☐ B. A runtime that replaces TF Serving
- ☐ C. A visualization tool for graphs only
- ☐ D. A GPU vendor library for GEMMs

Knowledge point: MLIR is a multi-level compiler infrastructure using dialects.

Slides: [Week 3 slides, p. 39](#) **Review PDF:** [All relevant slides, including other answer choices](#)

[Show answer & explanation](#)

Question 20

Which approach explicitly reduces cross-switch traffic by doing reductions within nodes first?

- ☐ A. Ring all-reduce
- ☐ B. Tree all-reduce
- ☐ C. Parameter server
- ☐ D. Hierarchical all-reduce

Knowledge point: Hierarchical all-reduce performs intra-node reduction before inter-node communication.

Slides: [Week 5 slides, p. 17](#) **Review PDF:** [All relevant slides, including other answer choices](#)

[Show answer & explanation](#)

Question 21

A personalization strategy discussed for FL is:

- ☐ A. Keep only the server head and freeze all client layers
- ☐ B. Upload all personal parameters for global averaging
- ☐ C. Split model: shared backbone aggregated globally + private head kept on-device
- ☐ D. Train only embeddings on device and discard the rest

Knowledge point: Federated personalization with a globally shared backbone and private local head.

Slides: [Week 5 slides, p. 44](#) **Review PDF:** [All relevant slides, including other answer choices](#)

[Show answer & explanation](#)

Question 22

For non-IID data, one aggregation tactic to improve convergence is:

- ☐ A. Cohort-wise aggregation: average within clusters then combine
- ☐ B. Weight only by network speed
- ☐ C. Discard minority cohorts
- ☐ D. Aggregate only the largest clients

Knowledge point: Cohort-wise aggregation for non-IID client data.

Slides: [Week 5 slides, pp. 45–46](#) **Review PDF:** [All relevant slides, including other answer choices](#)

[Show answer & explanation](#)

Question 23

In client-level DP for FL, what bounds a single client's influence prior to adding noise?

- ☐ A. Momentum correction
- ☐ B. Per-client norm clipping

- ☐ C. Homomorphic encryption
- ☐ D. Secure enclaves

Knowledge point: Client-level DP bounds influence through per-client norm clipping.

Slides: [Week 5 slides, p. 49](#) **Review PDF:** [All relevant slides, including other answer choices](#)

[Show answer & explanation](#)

Question 24

In DDP, gradient bucketing is primarily used to:

- ☐ A. Increase peak memory
- ☐ B. Reduce flops by pruning tensors
- ☐ C. Start all-reduce on ready buckets to overlap communication with compute
- ☐ D. Disable mixed precision safely

Knowledge point: DDP gradient bucketing overlaps all-reduce communication with backpropagation.

Slides: [Week 5 slides, p. 7](#) **Review PDF:** [All relevant slides, including other answer choices](#)

[Show answer & explanation](#)

Question 25

A versatile CPU/GPU collective library often used as a fallback in DL frameworks is:

- ☐ A. NCCL
- ☐ B. MPI
- ☐ C. Gloo
- ☐ D. NVSHMEM

Knowledge point: Gloo is a general CPU/GPU collective communication library.

Slides: [Week 5 slides, p. 17](#) **Review PDF:** [All relevant slides, including other answer choices](#)

[Show answer & explanation](#)

Question 26

On TPUs, a common practice to improve systolic array utilization is to:

- ☐ A. Use many tiny GEMMs
- ☐ B. Pad/align matrix dimensions to preferred multiples (e.g., 128)
- ☐ C. Disable XLA's layout selection
- ☐ D. Replace all convs with FFTs by default

Knowledge point: TPU systolic arrays and tile-aligned matrix dimensions.

Slides: [Week 6 slides, p. 12](#); [Week 6 slides, p. 16](#) **Review PDF:** [All relevant slides, including other answer choices](#)

[Show answer & explanation](#)

Question 27

For small messages, which reduce algorithm often has lower latency?

- ☐ A. Tree all-reduce
- ☐ B. Ring all-reduce
- ☐ C. All-to-all
- ☐ D. Broadcast

Knowledge point: Tree all-reduce reduces latency for small messages.

Slides: [Week 5 slides, p. 17](#) **Review PDF:** [All relevant slides, including other answer choices](#)

[Show answer & explanation](#)

Question 28

A graph-level transformation that avoids re-computing identical subgraphs is:

- ☐ A. Kernel inlining
- ☐ B. Layer freezing
- ☐ C. Gradient checkpointing
- ☐ D. Common subexpression elimination

Knowledge point: Common subexpression elimination avoids repeated graph computation.

Slides: [Week 3 slides, pp. 28–30](#) **Review PDF:** [All relevant slides, including other answer choices](#)

[Show answer & explanation](#)

Question 29

(Select all that apply) Code-size optimization tactics discussed include:

- ☐ A. Compile with -Os for size
- ☐ B. Link-time optimization (LTO)
- ☐ C. Link with --gc-sections after -ffunction-sections/-fdata-sections
- ☐ D. Enable verbose logging strings in release builds

Knowledge point: Embedded code-size reduction with -Os, LTO, and --gc-sections.

Slides: [Week 4 slides, p. 58](#) **Review PDF:** [All relevant slides, including other answer choices](#)

[Show answers & explanation](#)

Question 30

(Select all that apply) Benefits of computational graphs include:

- ☐ A. Automatic differentiation via chain rule on the DAG

- ☐ B. Global scheduling and kernel fusion opportunities
- ☐ C. Memory planning and buffer reuse
- ☐ D. Guaranteed elimination of all communication

Knowledge point: Computational graphs enable autodiff, global optimization, and memory planning.

Slides: [Week 1 slides, pp. 39–43](#) **Review PDF:** [All relevant slides, including other answer choices](#)

[Show answers & explanation](#)

Question 31

(Select all that apply) Communication-efficient FL updates may use:

- ☐ A. Quantization (e.g., 8/4/2-bit with per-tensor scales)
- ☐ B. Sparsification with error feedback
- ☐ C. Low-rank adapters (e.g., LoRA)
- ☐ D. Larger deltas sent as raw samples

Knowledge point: Communication-efficient federated updates: quantization, sparsification, and LoRA.

Slides: [Week 5 slides, p. 23](#) **Review PDF:** [All relevant slides, including other answer choices](#)

[Show answers & explanation](#)

Question 32

(Select all that apply) Systolic array-friendly GEMM tiling on TPUs involves:

- ☐ A. Partitioning A and B into fixed-size tiles
- ☐ B. Padding boundaries so tiles align
- ☐ C. Streaming A horizontally and B vertically across cells
- ☐ D. Random tile sizes per step to avoid regularity

Knowledge point: TPU systolic-array GEMM tiling and streamed operand movement.

Slides: [Week 3 slides, pp. 24–25](#); [Week 6 slides, p. 16](#) **Review PDF:** [All relevant slides, including other answer choices](#)

[Show answers & explanation](#)

Question 33

(Select all that apply) Robust FL aggregation under non-IID and noise may use:

- ☐ A. Trimmed means / coordinate-wise medians
- ☐ B. Update norm clipping
- ☐ C. Outlier filtering
- ☐ D. Always weighting solely by client RTT

Knowledge point: Robust federated aggregation for non-IID and noisy updates.

Slides: [Week 5 slides, pp. 45–46](#); [Week 5 slides, p. 49](#) **Review PDF:** [All relevant slides, including other](#)

answer choices

[Show answers & explanation](#)

Question 34

(Select all that apply) Reproducibility at scale recommendations include:

- ☐ A. Seed Python/NumPy/framework/CUDA RNGs
- ☐ B. Save RNG states and dataloader epoch/offset in checkpoints
- ☐ C. Pin environment versions (CUDA/cuDNN/NCCL, drivers, framework)
- ☐ D. Prefer nondeterministic kernels to speed up runs

Knowledge point: Large-scale reproducibility: RNG seeding, saved state, and pinned versions.**Slides:** [Week 5 slides, pp. 19–20](#) **Review PDF:** [All relevant slides, including other answer choices](#)[Show answers & explanation](#)

Question 35

(Select all that apply) In TFLM on MCUs, good memory practices are:

- ☐ A. Fixed tensor arena; fail fast if insufficient
- ☐ B. Avoid malloc/free in the hot path
- ☐ C. Preallocate/reuse buffers to avoid fragmentation
- ☐ D. Allocate per layer dynamically every inference

Knowledge point: TFLM memory planning with a fixed arena and reused buffers.**Slides:** [Week 1 slides, p. 55](#); [Week 6 slides, p. 23](#) **Review PDF:** [All relevant slides, including other answer choices](#)[Show answers & explanation](#)

Question 36

(Select all that apply) FSDP typically:

- ☐ A. All-gathers param shards just-in-time before compute
- ☐ B. Frees full params promptly after layer compute
- ☐ C. Reduce-scatters gradients after backward
- ☐ D. Requires full params on all ranks at all times

Knowledge point: FSDP just-in-time parameter all-gather and gradient reduce-scatter.**Slides:** [Week 5 slides, p. 18](#) **Review PDF:** [All relevant slides, including other answer choices](#)[Show answers & explanation](#)

Question 37

(Select all that apply) Pipeline parallelism knobs/trade-offs include:

- ☐ A. Increase microbatches b to shrink bubbles $((m-1)/b)$
- ☐ B. 1F1B cuts activation footprint but can add weight staleness
- ☐ C. Activation checkpointing reduces memory at extra compute
- ☐ D. GPipe always interleaves fwd/bwd per microbatch

Knowledge point: Pipeline microbatches, bubble reduction, 1F1B, and checkpoint trade-offs.

Slides: [Week 5 slides, pp. 11–12](#); [Week 2 slides, p. 40](#) **Review PDF:** [All relevant slides, including other answer choices](#)

[Show answers & explanation](#)

Question 38

(Select all that apply) Strategies to reduce serial fraction s or hide communication include:

- ☐ A. Overlap comm/compute via bucketization and fused kernels
- ☐ B. Lower-precision or compressed gradients with error feedback
- ☐ C. Increase local batch and consider partial/delayed sync (with care)
- ☐ D. Ignore stragglers and remove health checks

Knowledge point: Scaling strategies to overlap communication and computation.

Slides: [Week 5 slides, p. 7](#); [Week 5 slides, pp. 14–17](#); [Week 5 slides, p. 23](#) **Review PDF:** [All relevant slides, including other answer choices](#)

[Show answers & explanation](#)

Question 39

(Select all that apply) Memory-bandwidth-aware kernel practices:

- ☐ A. Align loop order with layout (NHWC/NCHW) to read contiguous memory
- ☐ B. Use in-place ops where safe to halve traffic
- ☐ C. Prefer many tiny kernel launches to improve “interleaving”
- ☐ D. Use memory pools/arenas to avoid runtime alloc/free overhead

Knowledge point: Memory-bandwidth-aware access, in-place operations, and allocator pools.

Slides: [Week 3 slides, pp. 25–26](#); [Week 6 slides, pp. 67–69](#) **Review PDF:** [All relevant slides, including other answer choices](#)

[Show answers & explanation](#)

Question 40

(Select all that apply) Elastic distributed training on world-size change:

- ☐ A. Recompute effective global batch; adjust LR accordingly
- ☐ B. Rebuild process groups and reshard data/checkpoints

- ☐ C. Keep loss-scaling consistent if precision unchanged
- ☐ D. Disable checkpoints to avoid mismatches

Knowledge point: Elastic training on world-size changes: batch/LR, process groups, and checkpoint resharding.

Slides: [Week 5 slides, p. 5](#); [Week 5 slides, pp. 16–20](#) **Review PDF:** [All relevant slides, including other answer choices](#)

[Show answers & explanation](#)

Answers & Explanations

Question 1

B — Reverse-mode AD is efficient for a scalar loss with many parameters (classic NN training).

[Back](#)

Question 2

C — $f(a+\epsilon b) = f(a) + f'(a) \cdot \epsilon \cdot b$ because $\epsilon^2=0$ (first-order Taylor).

[Back](#)

Question 3

D — TFLM uses a fixed tensor arena; no malloc/free in the hot path.

[Back](#)

Question 4

B — XLA fuses ops, picks layouts, and reuses buffers for speed/efficiency.

[Back](#)

Question 5

D — `--gc-sections` removes unused sections (with `-ffunction/-fdata-sections`).

[Back](#)

Question 6

C — Intra reduce-scatter → inter reduce-scatter → inter all-gather → intra all-gather is canonical.

[Back](#)

Question 7

D — Pairwise masks cancel; threshold recovery fixes dropouts in secure agg.

[Back](#)

Question 8

A — TF 2.x defaults to eager; tf.function can trace/compile hot paths.

[Back](#)

Question 9

C — TF 1.x used symbolic graphs executed in a Session (define-then-run).

[Back](#)

Question 10

B — FlatBuffers allow zero-copy deserialization with schema checks on MCUs.

[Back](#)

Question 11

D — Radios dominate energy; compute locally to avoid sending bytes.

[Back](#)

Question 12

C — Conv+BN+ReLU fusion is standard to cut memory traffic/launches.

[Back](#)

Question 13

A — For NHWC, reading c contiguously ($h \rightarrow w \rightarrow c$) improves cache/coalescing.

[Back](#)

Question 14

B — Sync all-reduce does not have weight staleness by design; it has per-step barriers and straggler sensitivity; it's equivalent to a large batch on one device (with same optimizer/schedule).

[Back](#)

Question 15

B — Async PS can increase throughput and tolerate slow clients (but risks staleness).

[Back](#)

Question 16

B — ZeRO-2 shards optimizer states and gradients to cut memory/traffic.

[Back](#)

Question 17

D — Reduce-scatter partitions work and accumulates partial reductions across ranks.

[Back](#)

Question 18

C — TFLM must fail fast if the arena is insufficient (determinism > silent fallback).

[Back](#)

Question 19

A — MLIR is a multi-level compiler framework (TF dialect → linalg → LLVM IR), not a runtime.

[Back](#)

Question 20

D — Hierarchical all-reduce reduces within nodes before inter-node, cutting cross-switch traffic.

[Back](#)

Question 21

C — Shared backbone aggregated globally + private head on device is a common personalization split in FL.

[Back](#)

Question 22

A — Aggregating within cohorts (clusters) then combining mitigates non-IID skew and speeds convergence.

[Back](#)

Question 23

B — Clip per-client update norms; then add calibrated noise for client-level DP guarantees (with an accountant).

[Back](#)

Question 24

C — Bucketing starts all-reduce early to overlap comm with remaining backprop compute.

[Back](#)

Question 25

C — Gloo is a versatile CPU/GPU collective library often used as a fallback (NCCL is GPU-centric).

[Back](#)

Question 26

B — Padding/alignment to TPU tile sizes improves systolic utilization.

[Back](#)

Question 27

A — Tree all-reduce tends to have lower latency for small messages than ring (fewer hops).

[Back](#)

Question 28

D — CSE avoids recomputing repeated subgraphs; complements fusion and scheduling passes.

[Back](#)

Question 29

A, B, C — -Os, LTO, and --gc-sections shrink binaries; verbose logs increase size.

[Back](#)

Question 30

A, B, C — Graphs enable autodiff, global optimization, and memory planning (not “no communication”).

[Back](#)

Question 31

A, B, C — Quantization, sparsification+error feedback, and low-rank adapters cut bytes; sending raw data is not used.

[Back](#)

Question 32

A, B, C — Fixed tiles with padding stream A right/B down; “random tile sizes” break regularity and utilization.

[Back](#)

Question 33

A, B, C — Fixed arena + no dynamic allocs in hot path + reuse buffers; don’t malloc per inference on MCUs.

[Back](#)

Question 34

A, B, C — Seed/Save/Pin ensure reproducibility; nondeterministic kernels break it.

[Back](#)

Question 35

A, B, C — These TFLM patterns keep memory deterministic and fragmentation-free.

[Back](#)

Question 36

A, B, C — FSDP all-gathers JIT, frees promptly, reduce-scatters grads; it doesn't require full params on all ranks always.

[Back](#)

Question 37

A, B, C — b increases shrink bubbles; 1F1B reduces memory but risks staleness; checkpointing trades compute for memory; GPipe is all-forward then all-backward.

[Back](#)

Question 38

A, B, C — Overlap comm/compute, compress gradients, and adjust batching/sync variants to lower s; ignoring stragglers harms stability.

[Back](#)

Question 39

A, B, D — Contiguous access + safe in-place ops + allocators/memory pools help bandwidth; many tiny launches hurt throughput.

[Back](#)

Question 40

A, B, C — Recompute effective batch & LR; rebuild groups/reshard; keep loss scaling consistent; never disable checkpoints.

[Back](#)